

Universidade Federal do Rio Grande – FURG

Centro de Ciências Computacionais

Disciplina: Sistemas para Internet 2

Web Proxy com Controle de Conteúdo

Autores: Lucas Derick Silva Pereira

Lincon Avila de Souza

Professor: Prof. Dr. André Prisco Vargas

Semestre: 2026/1

0 Sumário

1	Introdução	2
2	Relatório Técnico	2
2.1	Justificativa da Tecnologia Escolhida	2
2.2	Decisões de Implementação — O que foi difícil e o que surpreendeu	2
2.3	Como o Proxy Lida com HTTPS — Por que o Filtro Não Funciona	3
2.4	O que Aprendemos sobre o Protocolo HTTP	3
2.5	Uso de Inteligência Artificial	3
3	Código-Fonte – Fase 1: Configurações e Dados	4
3.1	config/blocked.json	4
3.2	config/words.json	4
4	Código-Fonte – Fase 2: Motor de Filtragem	5
4.1	proxy/config.py	5
4.2	proxy/filter.py	5
5	Código-Fonte – Fase 3: Interface Gráfica	6
5.1	templates/blocked.html	6
6	Código-Fonte – Fase 4: Documentação	7
6.1	README.md	7
7	Considerações Finais	7

1 Introdução

Neste relatório será documentado o desenvolvimento do **Web Proxy com Controle de Conteúdo**, trabalho prático da disciplina de Sistemas para Internet 2. O objetivo foi construir um servidor proxy funcional em Python, capaz de interceptar requisições HTTP, bloquear o acesso a domínios específicos e substituir palavras impróprias no conteúdo das páginas visitadas.

Ao longo do documento estão descritas as decisões técnicas tomadas durante o desenvolvimento, junto com o código-fonte de cada módulo implementado.

2 Relatório Técnico

2.1 Justificativa da Tecnologia Escolhida

Optamos por usar **Python 3** com o microframework **Flask** porque queríamos focar na lógica do proxy em si, sem precisar lidar manualmente com toda a parte de baixo nível de rede — como *parsing* de cabeçalhos HTTP, gerenciamento de *buffers* e concorrência em *sockets* TCP. O Flask abstraiu essa camada e nos deixou trabalhar diretamente na lógica da aplicação. Para fazer as requisições ao servidor de destino, usamos a biblioteca `requests`, que tornou o repasse das chamadas bem direto e legível.

2.2 Decisões de Implementação — O que foi difícil e o que surpreendeu

Dividimos o código em módulos separados para deixar cada responsabilidade bem definida:

- `proxy/config.py`: carrega os dois arquivos JSON de configuração usando caminhos absolutos gerados com `os.path.dirname`. Isso foi necessário porque durante os testes percebemos que o script lançava `FileNotFoundError` dependendo de qual diretório era usado para executá-lo — o caminho relativo simplesmente quebrava. Resolver isso com caminho absoluto baseado no próprio arquivo foi simples, mas não era algo que esperávamos ter que lidar.
- `proxy/filter.py`: aplica as substituições no HTML usando a biblioteca `re` com `re.IGNORECASE`. A parte mais trabalhosa aqui foi montar o dicionário de palavras de forma que as substituições não gerassem falsos positivos — por exemplo, palavras que são substrings de outras. Isso levou a alguns testes até chegar na configuração atual.
- `templates/blocked.html`: página de bloqueio renderizada pelo Jinja2, que exibe dinamicamente o domínio barrado. Essa parte foi mais tranquila — o Jinja2 integra bem com o Flask e bastou passar a variável pelo `render_template`.

O que mais surpreendeu foi perceber o quanto o proxy consegue ver do tráfego HTTP — cabeçalhos, corpo da resposta, tudo em texto claro. Isso deixou bem evidente por que o HTTPS se tornou padrão.

2.3 Como o Proxy Lida com HTTPS — Por que o Filtro Não Funciona

Durante os testes percebemos que o filtro de palavras não tem nenhum efeito em sites HTTPS, e entendemos o motivo: o protocolo usa **TLS/SSL** para cifrar todo o tráfego de ponta a ponta. Quando o navegador quer acessar um site HTTPS, ele envia ao proxy o método **CONNECT** para abrir um túnel direto até o servidor. A partir daí, o proxy é apenas um intermediário cego — só enxerga bytes cifrados passando pelo canal. Sem participar do *TLS Handshake* nem ter as chaves privadas do servidor, não há como acessar o HTML em texto claro para aplicar o `re.sub`.

O que seria necessário para funcionar: seria preciso implementar um **Proxy MITM** (*Man-in-the-Middle*). Nessa abordagem, o proxy gera uma Autoridade de Certificação (CA) local e cria certificados falsos para cada site acessado. Com a CA instalada no navegador como confiável, o proxy consegue terminar o TLS em dois canais separados — um com o cliente, outro com o servidor real — e assim lê o conteúdo em texto claro no meio. É exatamente o que ferramentas como o Burp Suite fazem. O custo disso é a complexidade de gestão de certificados e a necessidade de instalar a CA no cliente.

2.4 O que Aprendemos sobre o Protocolo HTTP

Implementar o proxy na prática trouxe algumas percepções que a teoria sozinha não deixava tão claras:

- **HTTP é texto puro e legível.** Ver os cabeçalhos das requisições e o HTML das respostas passando pelo proxy deixou evidente que, sem criptografia, qualquer intermediário na rede consegue ler e modificar o conteúdo livremente.
- **A separação entre cabeçalho e corpo.** Para repassar a resposta corretamente, precisamos entender que o proxy recebe primeiro os cabeçalhos HTTP (com metadados como **Content-Type** e **Content-Length**) e depois o corpo — e que modificar o corpo sem ajustar o **Content-Length** pode quebrar a resposta no navegador.
- **O papel do campo Host.** O navegador envia o domínio de destino no cabeçalho **Host**, e é isso que o proxy usa para saber para onde repassar a requisição — não a URL completa.
- **A diferença prática entre HTTP e HTTPS.** Antes do trabalho, HTTPS era só "HTTP com cadeado". Depois de ver o proxy travar diante de uma conexão **CONNECT** e não conseguir enxergar nada, ficou muito mais concreto o que a camada TLS realmente protege.

2.5 Uso de Inteligência Artificial

Usamos o **Gemini** como apoio em algumas partes do projeto:

- Geração e organização do dicionário `words.json` com os pares de substituição.
- Ajuda na construção da expressão regular do filtro.

- Estrutura inicial do template HTML da página de bloqueio.

O que foi gerado pela IA foi revisado e testado por nós antes de entrar no projeto.

3 Código-Fonte – Fase 1: Configurações e Dados

3.1 config/blocked.json

Lista de domínios que o proxy bloqueia. Qualquer requisição para esses endereços retorna a página de aviso ao invés de repassar a chamada.

Listing 1: Lista de domínios bloqueados

```
1 {
2   "bloqueados": [
3     "www.instagram.com",
4     "x.com",
5     "www.tiktok.com",
6     "www.linkedin.com",
7     "www.reddit.com",
8     "www.pinterest.com",
9     "www.snapchat.com",
10    "www.twitch.tv",
11    "www.netflix.com",
12    "www.spotify.com",
13    "www.discord.com",
14    "www.tumblr.com",
15    "www.vimeo.com",
16    "www.flickr.com",
17    "neverssl.com",
18    "eitc.org"
19  ]
20 }
```

3.2 config/words.json

Dicionário de substituições — cada chave é a palavra a ser filtrada e o valor é pelo que ela será trocada no HTML. O arquivo completo tem mais de 80 entradas; abaixo está um trecho representativo.

Listing 2: Trecho do dicionário de substituições

```
1 {
2   "arrombado": "desagradavel",
3   "babaca": "tolo",
4   "canalha": "pilantra",
5   "caralho": "caramba",
6   "foda-se": "deixa pra la",
7   "merda": "droga",
8   "porra": "droga",
9   "safado": "malandro",
10  "vagabundo": "desocupado",
11  "veado": "rapaz"
12 }
```

4 Código-Fonte – Fase 2: Motor de Filtragem

4.1 proxy/config.py

Responsável por carregar os dois arquivos JSON. Usamos `os.path.dirname` para montar o caminho de forma absoluta, o que resolve o problema de o script ser executado de diretórios diferentes.

```
1 import json
2 import os
3
4 def load_configs() -> tuple[list, dict]:
5     base_dir = os.path.dirname(os.path.abspath(
6         __file__))
7     blocked_path = os.path.join(base_dir, 'config', 'blocked.json')
8     words_path = os.path.join(base_dir, 'config', 'words.json')
9
10    lista_bloqueados = []
11    dicionario_palavroes = {}
12
13    try:
14        with open(blocked_path, 'r', encoding='utf-8') as f:
15            dados_bloqueados = json.load(f)
16            lista_bloqueados = dados_bloqueados.get('bloqueados',
17            [])
18    except FileNotFoundError:
19        print(f"Arquivo {blocked_path} nao encontrado.")
20
21    try:
22        with open(words_path, 'r', encoding='utf-8') as f:
23            dicionario_palavroes = json.load(f)
24    except FileNotFoundError:
25        print(f"Arquivo {words_path} nao encontrado.")
26
27    return lista_bloqueados, dicionario_palavroes
28
29 if __name__ == "__main__":
30     bloqueados, palavroes = load_configs()
31     print("Lista de bloqueados:", bloqueados)
32     print("Dicionario de palavroes:", palavroes)
```

Listing 3: Módulo de configuração

4.2 proxy/filter.py

Percorre o dicionário e aplica cada substituição no HTML recebido. A flag `re.IGNORECASE` garante que palavras em qualquer combinação de maiúsculas e minúsculas sejam capturadas. A função retorna o HTML modificado e um booleano indicando se alguma substituição foi feita.

```
1 import re
2 from proxy.config import load_configs
3
4 def filter_content(html: str) -> tuple[str, bool]:
```

```

5     _, words_dict = load_configs()
6
7     modified_html = html
8     was_filtered = False
9
10    for word, replacement in words_dict.items():
11        pattern = re.compile(re.escape(word), re.IGNORECASE)
12        if pattern.search(modified_html):
13            modified_html = pattern.sub(replacement, modified_html)
14            was_filtered = True
15
16    return modified_html, was_filtered

```

Listing 4: Módulo de filtragem de conteúdo

5 Código-Fonte – Fase 3: Interface Gráfica

5.1 templates/blocked.html

Página exibida quando o usuário tenta acessar um domínio bloqueado. O nome do domínio é passado dinamicamente pelo Flask via Jinja2 (`{{ dominio }}`).

Listing 5: Template da página de bloqueio

```

1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale
6         =1.0">
7     <title>Acesso Bloqueado</title>
8     <style>
9         * { margin: 0; padding: 0; box-sizing: border-box;
10            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif
11            ; }
12     body { background-color: #2c3e50; display: flex;
13            justify-content: center; align-items: center; height: 100
14            vh; }
15     .container { background: #fff; padding: 40px 50px;
16                 border-radius: 12px; max-width: 500px; width: 90%;
17                 border-top: 8px solid #e74c3c; text-align: center;
18                 }
19     h1 { color: #e74c3c; margin-bottom: 15px; }
20     .domain-blocked { background: #fce4e4; color: #c0392b;
21                     padding: 8px 15px; border-radius: 6px;
22                     font-weight: bold; font-size: 18px; }
23 </style>
24 </head>
25 <body>
26     <div class="container">
27         <div class="proxy-name">Web Proxy Server - SI2</div>
28         <h1>Acesso Restrito</h1>
29         <p>A politica de segurança deste proxy impediu a navegação
30         para o site solicitado.</p>
31         <div class="domain-blocked">{{ dominio }}</div>
32         <div class="footer">

```

```
29     Sistemas para Internet 2 &bull; Controle de Conteudo
30     </div>
31 </div>
32 </body>
33 </html>
```

6 Código-Fonte – Fase 4: Documentação

6.1 README.md

Instruções de instalação e execução do servidor proxy.

```
1 # Web Proxy com Controle de Conteudo
2
3 Servidor Web Proxy didatico em Python/Flask para SI2.
4
5 ## Pre-requisitos
6 - Python 3.x instalado no sistema.
7
8 ## Instalacao e Execucao
9
10 1. Criar e ativar ambiente virtual (Linux/WSL):
11     python3 -m venv venv
12     source venv/bin/activate
13
14 2. Instalar dependencias:
15     pip install -r requirements.txt
16
17 3. Iniciar o servidor:
18     python3 app.py
19
20 O servidor ficara ativo em http://localhost:5000
21
22 ## Como utilizar
23
24 Acesso normal:          http://localhost:5000/http://www.pudim.com.
25                          br
26 Dominio bloqueado:      http://localhost:5000/http://www.instagram.
27                          com
28
29 Nota: O filtro atua apenas sobre trafico HTTP limpo.
30       Sites HTTPS enviam dados encriptados, impossibilitando
31       a substituicao de palavras pelo proxy.
```

Listing 6: README – instruções de uso

7 Considerações Finais

O trabalho nos deu uma visão prática de como um proxy HTTP funciona por dentro. Implementar o bloqueio de domínios e o filtro de palavras deixou claro como o proxy tem acesso total ao conteúdo em HTTP — e ao mesmo tempo evidenciou a limitação

real quando se trata de HTTPS, onde esse controle simplesmente não é possível sem uma infraestrutura de interceptação mais complexa.